

FastAPI Department Module

A Detailed Learning Report

Project	PM-SYS/BE — Project & Budget Tracking System
Module Studied	departments
Report Type	Detailed Technical Learning Report
Author	Sanskar Debnath
Focus	FastAPI layered architecture, from request to database

This report explains the uploaded FastAPI **departments** module from the ground up: how a request travels from the client to PostgreSQL and back, what each layer of the codebase is responsible for, and a reliable development order that can be reused for future modules in the system.

Submitted as part of academic coursework

Table of Contents

1. What this module is trying to build	8
2. Architecture overview	9
Responsibility of each layer	9
3. Recommended folder structure	11
4. Which file should be created first?	12
Why this order works	12
5. Complete request flow	14
5.1 Where does `department_data` come from?	14
5.2 Where does `db` come from?	15
6. Controller explained line by line	17
6.1 Imports	17
6.2 Router creation	18
6.3 Route decorator	18
6.4 Function parameters	19
6.5 Calling the service	19
6.6 Returning a dictionary	20
7. Schema or DTO explained	21
7.1 What is a Pydantic schema?	21
7.2 Is it a schema or a DTO?	21
7.3 Field explanation	21
7.4 What happens when input is invalid?	22
7.5 Naming concern	22
8. Database configuration explained	24

8.1 `os`	24
8.2 `Path`	24
8.3 Loading `.env`	24
8.4 Creating the database URL	25
8.5 Creating the engine	25
8.6 Session factory	26
8.7 Declarative base class	27
8.8 `get_db`	27
9. SQLAlchemy model explained	29
9.1 Abstract base model	29
9.2 UUID column	29
9.3 Table name and schema	30
9.4 Primary key	31
9.5 Department name	31
9.6 Budget	32
9.7 Creation timestamp	32
10. Repository explained	34
10.1 Why have a repository?	34
10.2 `@staticmethod`	34
10.3 Parameters	35
10.4 Meaning of `-> DBDepartment`	35
10.5 Creating the ORM object	36
10.6 Returning the model	37
11. Service explained	38
11.1 What is the service layer for?	38
11.2 Why not call the repository directly from the controller?	38
11.3 Example future validation	39
12. Application entry point explained	40
12.1 Creating the app	40

12.2 Including the Department router	40
12.3 CORS	40
12.4 Health endpoint	41
12.5 Placeholder controller scanner	41
13. Meaning of important Python syntax	42
13.1 Colon in a parameter	42
13.2 Equals sign in a parameter	42
13.3 Arrow	42
13.4 Decorator	42
13.5 Class inheritance	42
13.6 `yield`	42
13.7 Lambda	43
13.8 Attribute access	43
13.9 Keyword arguments	43
14. What `db.add`, `db.commit`, and `db.refresh` do	44
14.1 `db.add(new_department)`	44
14.2 `db.commit()`	44
14.3 `db.refresh(new_department)`	45
14.4 Difference between `flush` and `commit`	45
15. Detailed execution example	46
Phase 1: FastAPI route matching	46
Phase 2: Pydantic validation	46
Phase 3: Dependency injection	46
Phase 4: Controller	47
Phase 5: Service	47
Phase 6: Repository object construction	47
Phase 7: Session tracking	47
Phase 8: Database transaction	47
Phase 9: Reloading generated values	47
Phase 10: Return through layers	48

Phase 11: HTTP response	48
Phase 12: Session cleanup	48
16. Spring Boot comparison	49
17. Problems and improvement opportunities in the uploaded code	51
17.1 Duplicate imports in the controller	51
17.2 Duplicate and unused imports in the model	51
17.3 Unused imports	51
17.4 Import path inconsistency	52
17.5 Module path assumption	52
17.6 Auto-increment ID versus DTO-provided ID	53
17.7 No response model	53
17.8 No rollback on commit failure	53
17.9 No duplicate-department check	54
17.10 Department name nullable	54
17.11 Budget validation	54
17.12 Synchronous versus asynchronous description	54
17.13 Database schema creation	54
17.14 `create_all` is commented out	55
17.15 Controller variable not used	55
17.16 Route naming	55
18. A cleaner learning-oriented implementation	56
18.1 `core/database.py`	56
18.2 `models/DepartmentTables.py`	56
18.3 `schemas/DepartmentSchema.py`	57
18.4 `repositories/DepartmentRepo.py`	58
18.5 `services/DepartmentService.py`	59
18.6 `controllers/DepartmentController.py`	60
18.7 `main.py`	61
19. How to create another module in the future	63

Step 1: Identify the database fields	63
Step 2: Create the model	63
Step 3: Create schemas	63
Step 4: Create repository	64
Step 5: Create service	64
Step 6: Create controller	64
Step 7: Register router	65
Step 8: Create migration	65
Step 9: Test every layer	65
20. Testing checklist	66
20.1 Successful creation	66
20.2 Missing name	66
20.3 Negative budget	66
20.4 Duplicate name	66
20.5 Database unavailable	67
20.6 Generated fields	67
20.7 Session cleanup	67
21. Common errors and how to understand them	68
`ModuleNotFoundError: No module named 'modules'`	68
`ModuleNotFoundError: No module named 'core'`	68
`relation "departments.departments" does not exist`	68
`schema "departments" does not exist`	68
`duplicate key value violates unique constraint`	69
`current transaction is aborted`	69
`422 Unprocessable Entity`	69
`500 Internal Server Error`	69
22. Final mental model	70
Recommended development sequence summary	71

Visual Overview

The two figures below summarise the module's layered architecture and the path a single request takes through the system, before the chapter-by-chapter explanation begins.

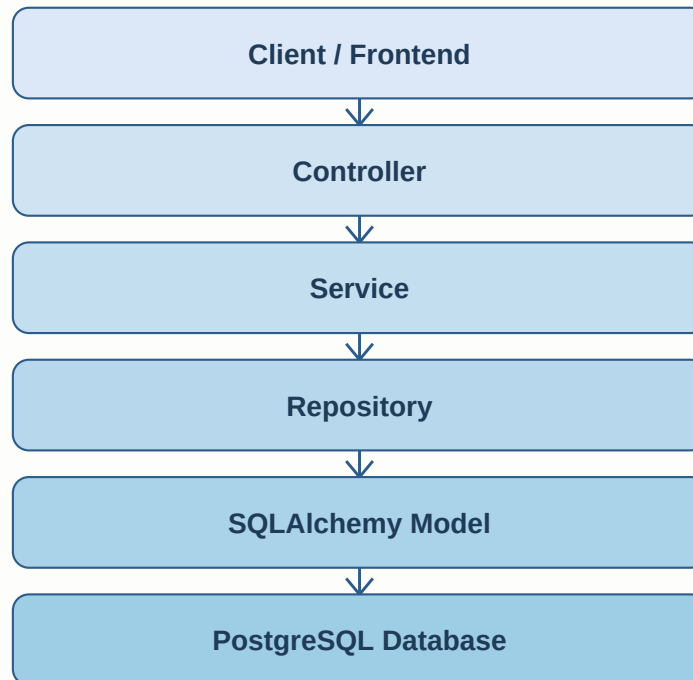


Figure 1: Layered architecture of the departments module.

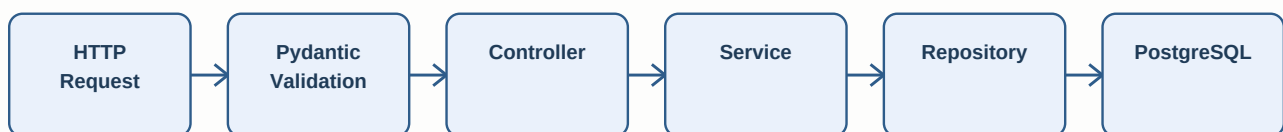


Figure 2: End-to-end request flow, from HTTP request to database write.

1. What this module is trying to build

The Department module creates an API endpoint that receives department information and stores it in PostgreSQL.

A frontend or API client sends a request such as:

```
POST /v1/department/add-department
Content-Type: application/json
```

```
{
  "did": 101,
  "dname": "Information Technology",
  "dbudget": 2500000.00
}
```

The application then performs these operations:

- FastAPI receives the HTTP request.
- Pydantic validates the JSON body.
- FastAPI creates a database session through `get_db`.
- The controller calls the service.
- The service calls the repository.
- The repository creates a SQLAlchemy object.
- SQLAlchemy inserts the row into PostgreSQL.
- The API returns an HTTP `201 Created` response.

The important idea is that every layer has a separate responsibility.

2. Architecture overview

The uploaded project follows a layered architecture:

```
Client / Frontend
  |
  v
Controller
  |
  v
Service
  |
  v
Repository
  |
  v
SQLAlchemy Model
  |
  v
PostgreSQL Database
```

The response comes back in the reverse direction:

```
PostgreSQL
  |
  v
Repository
  |
  v
Service
  |
  v
Controller
  |
  v
Client / Frontend
```

Responsibility of each layer

Layer	Main responsibility	Should know about
Controller	HTTP request and HTTP response	FastAPI, DTO/schema, service
Service	Business rules and use-case logic	Repository, DTO/schema
Repository	Database operations	SQLAlchemy session, model
Model	Database table structure	SQLAlchemy

Layer	Main responsibility	Should know about
Schema/DTO	Request and response data structure	Pydantic
Database configuration	Engine, sessions, base class	SQLAlchemy and environment variables
<code>main.py</code>	Create app and connect routers	FastAPI routers and middleware

A good rule is:

The controller should not write SQL, and the repository should not decide HTTP status codes.

That separation makes the project easier to test, maintain, and extend.

3. Recommended folder structure

A consistent structure for the project would be:

```
project_root/
|
|--- main.py
|--- .env
|--- .gitignore
|
|--- core/
|   |--- __init__.py
|   +-- database.py
|
|--- modules/
|   |--- __init__.py
|   |
|   +-- departments/
|       |--- __init__.py
|       |
|       |--- controllers/
|       |   |--- __init__.py
|       |   +-- DepartmentController.py
|       |
|       |--- models/
|       |   |--- __init__.py
|       |   +-- DepartmentTables.py
|       |
|       |--- repositories/
|       |   |--- __init__.py
|       |   +-- DepartmentRepo.py
|       |
|       |--- schemas/
|       |   |--- __init__.py
|       |   +-- DepartmentSchema.py
|       |
|       +-- services/
|           |--- __init__.py
|           +-- DepartmentService.py
|
|--- tests/
|   +-- departments/
|       +-- test_department.py
```

The `__init__.py` files tell Python that the directories can be treated as packages.

In modern Python, namespace packages can sometimes work without `__init__.py`, but keeping these files is useful for clarity and predictable imports.

4. Which file should be created first?

For a new module such as Department, the safest development order is:

1. Database configuration
2. SQLAlchemy model
3. Pydantic schemas/DTOs
4. Repository
5. Service
6. Controller/router
7. Register router in main.py
8. Migration/database setup
9. Tests

For the Department module specifically:

```
core/database.py
    v
models/DepartmentTables.py
    v
schemas/DepartmentSchema.py
    v
repositories/DepartmentRepo.py
    v
services/DepartmentService.py
    v
controllers/DepartmentController.py
    v
main.py
```

Your proposed order is fundamentally correct:

```
Model -> Schema -> Repository -> Service -> Controller
```

However, the database foundation must already exist before the model can import `Base`, and `main.py` must finally include the router.

Why this order works

Step 1: Database configuration

The model needs `Base`, and the repository needs `Session`.

Step 2: Model

The repository must know which SQLAlchemy class represents the database table.

Step 3: Schema/DTO

The controller needs a Pydantic class to validate incoming data. The service and repository may also use it as an input type.

Step 4: Repository

The repository depends on both the model and the input schema.

Step 5: Service

The service depends on the repository.

Step 6: Controller

The controller depends on the service, DTO, and database dependency.

Step 7: `main.py`

The application imports and registers the router after the router exists.

This creates a one-direction dependency chain and reduces circular-import problems.

5. Complete request flow

The uploaded controller contains:

```
@router.post(
    "/add-department",
    tags=["Add-department"],
    status_code=status.HTTP_201_CREATED
)
def create_department(
    department_data: DepartmentDTO,
    db: Session = Depends(get_db)
):
    new_department = DepartmentService.add_department(db, department_data)
    return {"message": "Department Added Successfully"}
```

There are two parameters:

```
department_data: DepartmentDTO
```

and:

```
db: Session = Depends(get_db)
```

They are obtained in two different ways.

5.1 Where does `department_data` come from?

`department_data` comes from the HTTP request body.

Because `DepartmentDTO` is a Pydantic model, FastAPI understands that the client must send JSON matching that structure.

Given:

```
class DepartmentDTO(BaseModel):
    did: int
    dname: str
    dbudget: Decimal
```

FastAPI expects:

```
{
  "did": 101,
  "dname": "Information Technology",
  "dbudget": 2500000.00
}
```

FastAPI performs this work before entering the controller function:

- Reads the JSON body.
- Checks that `did` can be treated as an integer.
- Checks that `dname` can be treated as a string.
- Checks that `dbudget` can be represented as a `Decimal`.
- Creates a `DepartmentDTO` object.
- Passes that object to `create_department`.

Therefore, inside the controller:

```
department_data.did  
department_data.dname  
department_data.dbudget
```

are normal Python attribute accesses.

In Spring Boot terminology, this is similar to:

```
public ResponseEntity<?> createDepartment(  
    @RequestBody DepartmentDTO departmentData  
)
```

5.2 Where does `db` come from?

`db` is not received from the JSON body.

This expression:

```
db: Session = Depends(get_db)
```

tells FastAPI:

Before calling this endpoint, execute the `get_db` dependency and inject the yielded value into `db`.

The `get_db` function is:

```
def get_db() -> Generator[Session, None, None]:  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

Execution:

- `SessionLocal()` creates a new SQLAlchemy session.
- `yield db` gives the session to the controller.
- The controller, service, and repository use the same session.

- After the request finishes, execution returns to `get_db`.
- The `finally` block closes the session.

This is dependency injection.

It is conceptually similar to Spring injecting a repository or an `EntityManager`, although the lifecycle implementation is different.

6. Controller explained line by line

Uploaded controller:

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session

from fastapi import APIRouter, Depends, HTTPException, status
from core.database import get_db
from modules.departments.services.DepartmentService import DepartmentService
from modules.departments.schemas.DepartmentSchema import DepartmentDTO

router = APIRouter(prefix="/v1/department")

@router.post("/add-department", tags=["Add-department"], status_code=status.HTTP_201_CREATED)
def create_department(department_data: DepartmentDTO, db: Session = Depends(get_db)):
    new_department = DepartmentService.add_department(db, department_data)
    return {"message": "Department Added Successfully"}
```

6.1 Imports

```
from fastapi import APIRouter, Depends, status
```

- **APIRouter** creates a group of related endpoints.
- **Depends** activates FastAPI dependency injection.
- **status** provides readable HTTP status constants.

Instead of writing:

```
status_code=201
```

the code uses:

```
status_code=status.HTTP_201_CREATED
```

Both mean the same thing, but the second form is easier to understand.

```
from sqlalchemy.orm import Session
```

Session represents a unit of interaction with the database. It tracks objects, executes SQL, commits transactions, rolls back transactions, and refreshes ORM objects.

```
from core.database import get_db
```

This imports the dependency that creates and closes one session for each request.

```
from modules.departments.services.DepartmentService import DepartmentService
```

The controller calls the business/service layer instead of directly calling the repository.

```
from modules.departments.schemas.DepartmentSchema import DepartmentDTO
```

This imports the Pydantic model used to validate the request body.

6.2 Router creation

```
router = APIRouter(prefix="/v1/department")
```

Every endpoint declared with this router starts with:

```
/v1/department
```

The endpoint decorator adds:

```
/add-department
```

Therefore, the final route becomes:

```
/v1/department/add-department
```

6.3 Route decorator

```
@router.post(  
    "/add-department",  
    tags=["Add-department"],  
    status_code=status.HTTP_201_CREATED  
)
```

The `@` syntax applies a decorator.

The decorator registers the function below it as an HTTP `POST` endpoint.

```
"/add-department"
```

This is the endpoint path relative to the router prefix.

```
tags=["Add-department"]
```

This groups the endpoint in Swagger/OpenAPI documentation.

A clearer tag would usually be:

```
tags=["Departments"]
```

because tags normally represent a resource group, not one particular operation.

```
status_code=status.HTTP_201_CREATED
```

If the function completes successfully, FastAPI sends HTTP status 201.

201 Created is appropriate when a new database resource has been created.

6.4 Function parameters

```
def create_department(  
    department_data: DepartmentDTO,  
    db: Session = Depends(get_db)  
):
```

`department_data: DepartmentDTO`

- Variable name: `department_data`
- Type hint: `DepartmentDTO`
- Source: request body
- Created by: FastAPI/Pydantic

`db: Session = Depends(get_db)`

- Variable name: `db`
- Type hint: `Session`
- Default expression: `Depends(get_db)`
- Source: dependency injection
- Created by: `SessionLocal()` inside `get_db`

6.5 Calling the service

```
new_department = DepartmentService.add_department(db, department_data)
```

This calls a static method on the service class.

Arguments:

<code>db</code>	-> active SQLAlchemy session
<code>department_data</code>	-> validated DepartmentDTO object

Return value:

DBDepartment object

The returned object is stored in:

`new_department`

However, the uploaded controller does not use `new_department` afterward. It only returns a message.

That works, but the API cannot tell the client the created department ID, UUID, name, budget, or creation time.

6.6 Returning a dictionary

```
return {"message": "Department Added Successfully"}
```

FastAPI automatically serializes the Python dictionary into JSON:

```
{  
  "message": "Department Added Successfully"  
}
```

7. Schema or DTO explained

Uploaded schema:

```
from pydantic import BaseModel
from datetime import datetime
from decimal import Decimal

class DepartmentDTO(BaseModel):
    did: int
    dname: str
    dbudget: Decimal
```

7.1 What is a Pydantic schema?

A Pydantic model describes data entering or leaving the API.

It is not the database table.

This distinction is critical:

```
Pydantic model    -> API data validation
SQLAlchemy model  -> Database table mapping
```

7.2 Is it a schema or a DTO?

In FastAPI documentation, it is often called a **schema** or **Pydantic model**.

In Spring Boot terminology, it is similar to a **DTO**.

Both descriptions are understandable:

```
DepartmentDTO = API input object
DBDepartment  = database entity/model
```

7.3 Field explanation

```
did: int
```

The API requires an integer field named `did`.

```
dname: str
```

The API requires a string field named `dname`.

```
dbudget: Decimal
```

The API requires a numeric value that Pydantic converts to `Decimal`.

`Decimal` is generally better than `float` for money because binary floating-point values can introduce precision errors.

For example, floating-point arithmetic may conceptually behave like:

```
0.1 + 0.2 != exactly 0.3
```

`Decimal` is designed for exact decimal arithmetic.

7.4 What happens when input is invalid?

Suppose the client sends:

```
{
  "did": "abc",
  "dname": "IT",
  "dbudget": 1000
}
```

"abc" cannot be converted into an integer, so FastAPI returns a `422 Unprocessable Entity` response before the controller executes.

This means the service and repository receive already-validated data.

7.5 Naming concern

The field names work, but they are abbreviated:

```
did
dname
dbudget
```

More descriptive names are easier to understand:

```
class DepartmentCreate(BaseModel):
    department_name: str
    department_budget: Decimal
```

A create request usually should not contain `department_id` when the database column is auto-incremented.

Your model says:

```
department_id = Column(
    Integer,
    primary_key=True,
    autoincrement=True
)
```

but the DTO requires:

```
did: int
```

and the repository manually assigns it:

```
department_id=department_data.did
```

These two designs conflict conceptually.

Choose one:

Option A: Database generates the ID

Recommended:

```
class DepartmentCreate(BaseModel):  
    department_name: str  
    department_budget: Decimal
```

Repository:

```
new_department = DBDepartment(  
    department_name=department_data.department_name,  
    department_budget=department_data.department_budget  
)
```

Option B: Client supplies the ID

Then `autoincrement=True` is less meaningful, and you must prevent duplicate IDs carefully.

For most APIs, Option A is cleaner.

8. Database configuration explained

Uploaded database configuration:

```
import os
from pathlib import Path
from typing import Generator

from dotenv import load_dotenv
from sqlalchemy import URL, create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
```

8.1 `os`

```
import os
```

Used to read environment variables:

```
os.getenv("DB_USER", "postgres")
```

8.2 `Path`

```
from pathlib import Path
```

`Path` provides object-oriented file path handling.

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

Breaking it down:

- `__file__` is the current Python file path.
- `.resolve()` converts it to an absolute path.
- `.parents[2]` moves upward through parent directories.

This line assumes that `database.py` is located deep enough inside the project, such as:

```
project_root/app/core/database.py
```

If `database.py` is directly under the project root, `.parents[2]` may point too far upward. The correct number depends on the final folder location.

8.3 Loading `.env`

```
load_dotenv(PROJECT_ROOT / ".env")
```


This loads variables from:

```
project_root/.env
```

Example `.env`:

```
DB_USER=postgres
DB_PASSWORD=your_password
DB_HOST=localhost
DB_PORT=5432
DB_NAME=project_management
```

The `.env` file should not be committed to Git because it may contain passwords.

Your `.gitignore` includes:

```
.env
```

which is correct.

8.4 Creating the database URL

```
DATABASE_URL = URL.create(
    drivename="postgresql+psycopg2",
    username=os.getenv("DB_USER", "postgres"),
    password=os.getenv("DB_PASSWORD"),
    host=os.getenv("DB_HOST", "localhost"),
    port=int(os.getenv("DB_PORT", "5432")),
    database=os.getenv("DB_NAME", "project_management"),
)
```

This creates a structured SQLAlchemy URL.

Equivalent conceptual URL:

```
postgresql+psycopg2://postgres:password@localhost:5432/project_management
```

Using `URL.create` is safer than manually joining a string, especially when passwords contain special characters.

8.5 Creating the engine

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
```

The engine manages:

- Database connectivity
- Connection pooling
- SQL execution infrastructure

`pool_pre_ping=True` checks whether a pooled connection is still alive before using it. It helps recover from stale or closed database connections.

Important: this is a **synchronous** SQLAlchemy engine.

Although the `main.py` comment says "async ORM," the uploaded database code uses:

```
create_engine
Session
sessionmaker
psycopg2
```

These are synchronous components.

An asynchronous setup would use components such as:

```
create_async_engine
AsyncSession
async_sessionmaker
asyncpg
```

8.6 Session factory

```
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False,
    expire_on_commit=False,
)
```

`SessionLocal` is not one session. It is a factory that creates sessions.

Calling:

```
db = SessionLocal()
```

creates an actual session.

`bind=engine`

The session uses the configured engine.

`autocommit=False`

Transactions are not automatically committed. The code must call:

```
db.commit()
```

`autoflush=False`

SQLAlchemy will not automatically flush pending changes at every possible point. The application controls transaction behavior more explicitly.

```
expire_on_commit=False
```

After commit, ORM object attributes remain available without immediately being marked expired.

8.7 Declarative base class

```
class Base(DeclarativeBase):  
    pass
```

Every SQLAlchemy model inherits from `Base`.

Example:

```
class DBDepartment(Base):  
    ...
```

This registers model metadata with SQLAlchemy.

8.8 `get_db`

```
def get_db() -> Generator[Session, None, None]:  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

The type annotation:

```
Generator[Session, None, None]
```

means:

```
Yielded value: Session  
Value sent into generator: None  
Final return value: None
```

The endpoint receives the yielded `Session`.

The `finally` block executes even when:

- The endpoint succeeds
- Validation after dependency creation fails
- The service raises an exception
- The repository raises a database exception

This ensures that the database session is closed.

Closing a session does not delete the database. It releases its connection and resources.

9. SQLAlchemy model explained

Uploaded model:

```
class DBDepartmentBase(Base):
    __abstract__ = True

    department_uuid = Column(
        UUID(as_uuid=True),
        default=py_uuid.uuid4,
        unique=True,
        nullable=False
    )
```

and:

```
class DBDepartment(DBDepartmentBase):
    __tablename__ = "departments"
    __table_args__ = {"schema": "departments"}
```

9.1 Abstract base model

```
class DBDepartmentBase(Base):
```

This class inherits from SQLAlchemy's declarative base.

```
    __abstract__ = True
```

This tells SQLAlchemy:

Do not create a table directly for `DBDepartmentBase`.

It exists only to provide shared columns or behavior to child models.

Here, it contributes:

```
    department_uuid
```

to `DBDepartment`.

9.2 UUID column

```
    department_uuid = Column(
        UUID(as_uuid=True),
        default=py_uuid.uuid4,
        unique=True,
        nullable=False
    )
```

```
UUID(as_uuid=True)
```

The PostgreSQL column uses the UUID data type.

`as_uuid=True` tells SQLAlchemy to work with Python `uuid.UUID` objects instead of plain strings.

```
default=py_uuid.uuid4
```

A new UUID is generated when a row is inserted.

Notice there are no parentheses:

```
default=py_uuid.uuid4
```

This passes the function itself so SQLAlchemy can call it separately for every new row.

This would be wrong:

```
default=py_uuid.uuid4()
```

because it would execute immediately when the model class is loaded.

```
unique=True
```

No two departments may have the same UUID.

```
nullable=False
```

The column cannot contain SQL `NULL`.

9.3 Table name and schema

```
__tablename__ = "departments"
```

The table name is:

```
departments
```

```
__table_args__ = {"schema": "departments"}
```

The PostgreSQL schema is also:

```
departments
```

The fully qualified table becomes:

```
departments.departments
```

That is valid, but it may be confusing because the schema and table have the same name.

For example, a clearer design could be:

```
organization.departments
```

or simply use the default `public` schema:

```
public.departments
```

Also, PostgreSQL must already contain the schema before the table is created, unless a migration explicitly creates it.

Example:

```
CREATE SCHEMA IF NOT EXISTS departments;
```

9.4 Primary key

```
department_id = Column(  
    Integer,  
    primary_key=True,  
    autoincrement=True,  
    index=True  
)
```

`Integer`

The column stores an integer.

`primary_key=True`

This uniquely identifies each row.

Primary keys are automatically non-null and unique.

`autoincrement=True`

PostgreSQL generates the next ID.

`index=True`

Creates an index for faster lookup.

A primary key is already indexed by PostgreSQL, so adding `index=True` is usually redundant for the primary key.

9.5 Department name

```
department_name = Column(  
    String(150),  
    index=True  
)
```

`String(150)`

Maximum database length is 150 characters.

`index=True`

Creates an index to improve searches by department name.

There is no:

```
nullable=False
```

Therefore the database may allow a null department name unless SQLAlchemy/database defaults or migrations say otherwise.

For a required department name, use:

```
nullable=False
```

You may also want:

```
unique=True
```

if duplicate department names are forbidden by business rules.

9.6 Budget

```
department_budget = Column(
    Numeric(15, 2),
    nullable=False,
    default=0
)
```

`Numeric(15, 2)` means:

```
Total digits: 15
Digits after decimal point: 2
```

Example valid conceptual value:

```
1234567890123.45
```

This type generally maps to Python `Decimal`.

`nullable=False` means a budget must exist.

`default=0` supplies zero if the application does not provide a value.

You may want a validation rule that prevents negative budgets.

9.7 Creation timestamp


```
department_created_at = Column(
    DateTime(timezone=True),
    default=lambda: datetime.now(timezone.utc),
    nullable=False
)
```

`DateTime(timezone=True)`

Stores a timezone-aware datetime.

`default=lambda: datetime.now(timezone.utc)`

Creates the current UTC time for each new object.

The `lambda` delays execution until row creation time.

Application-side versus database-side default

This is an application-side default.

A database-side alternative is:

```
server_default=func.now()
```

With `server_default`, PostgreSQL itself supplies the value.

10. Repository explained

Uploaded repository:

```
class DepartmentRepo:
    @staticmethod
    def add_department(
        db: Session,
        department_data: DepartmentDTO
    ) -> DBDepartment:
        new_department = DBDepartment(
            department_id=department_data.did,
            department_name=department_data.dname,
            department_budget=department_data.dbudget
        )

        db.add(new_department)
        db.commit()
        db.refresh(new_department)
        return new_department
```

10.1 Why have a repository?

The repository isolates database operations.

Without a repository, the controller might contain:

```
new_department = DBDepartment(...)
db.add(new_department)
db.commit()
```

That mixes HTTP handling with persistence logic.

The repository keeps the controller cleaner and makes future database changes easier.

10.2 @staticmethod

```
@staticmethod
```

means the method does not receive:

```
self
```

or:

```
cls
```

It can be called directly on the class:

```
DepartmentRepo.add_department(db, department_data)
```

instead of:

```
repo = DepartmentRepo()  
repo.add_department(db, department_data)
```

The method behaves like a regular function organized inside a class namespace.

10.3 Parameters

```
db: Session
```

`db` should be a SQLAlchemy `Session`.

```
department_data: DepartmentDTO
```

`department_data` should be a validated `DepartmentDTO`.

Type hints help:

- Developers
- IDE autocomplete
- Static type checkers
- Documentation
- Code readability

Python normally does not enforce these type hints automatically at runtime.

10.4 Meaning of `-> DBDepartment`

```
def add_department(...) -> DBDepartment:
```

The arrow introduces a **return type annotation**.

It means:

This function is expected to return a `DBDepartment` object.

It does not send data anywhere.

It does not convert the return value.

It does not itself perform a database operation.

It is a promise or documentation for developers and tools.

Example:

```
def add(a: int, b: int) -> int:
    return a + b
```

Here:

```
a: int      -> expected parameter type
b: int      -> expected parameter type
-> int      -> expected return type
```

Likewise:

```
def add_department(...) -> DBDepartment:
```

means the expected result is:

```
DBDepartment
```

The actual returned object comes from:

```
return new_department
```

If the function returned a string instead, ordinary Python might still run, but the annotation would be inaccurate. A type checker such as mypy or an IDE could warn about it.

So the arrow is better described as annotation syntax, not a normal arithmetic or flow operator.

10.5 Creating the ORM object

```
new_department = DBDepartment(
    department_id=department_data.did,
    department_name=department_data.dname,
    department_budget=department_data.dbudget
)
```

At this moment, Python creates an ORM object in memory.

No SQL **INSERT** has necessarily happened yet.

Its SQLAlchemy state is initially similar to:

```
Transient
```

After **db.add**, it becomes:

```
Pending
```

After a successful flush/commit, it becomes:

```
Persistent
```

10.6 Returning the model

```
return new_department
```

The repository returns the saved SQLAlchemy object to the service.

The service returns it to the controller.

The controller currently ignores its details and returns only a message.

11. Service explained

Uploaded service:

```
class DepartmentService:
    @staticmethod
    def add_department(
        db: Session,
        department_data: DepartmentDTO
    ) -> DBDepartment:
        return DepartmentRepo.add_department(db, department_data)
```

11.1 What is the service layer for?

The service layer contains business rules.

Examples of future business rules:

- Department name must be unique.
- Budget cannot be negative.
- Only an administrator may create a department.
- Budget cannot exceed an organization-level limit.
- The department code must follow a format.
- Audit history must be recorded.
- A notification must be sent after creation.

The current service simply passes data to the repository:

```
return DepartmentRepo.add_department(db, department_data)
```

This is acceptable at the beginning because it preserves the architecture. Later, business logic can be added without changing the controller.

11.2 Why not call the repository directly from the controller?

You technically can, but then business logic may become scattered among controllers.

Recommended:

```
Controller -> Service -> Repository
```

Avoid:

```
Controller -> Repository
```

when the project intentionally uses a service layer.

11.3 Example future validation

```
if department_data.department_budget < 0:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="Department budget cannot be negative"
    )
```

This belongs more naturally in the service than in the repository because it is a business rule, not a low-level database operation.

12. Application entry point explained

Important parts of `main.py`:

```
from fastapi import FastAPI
from modules.departments.controllers.DepartmentController import (
    router as department_router
)
```

12.1 Creating the app

```
app = FastAPI(
    title="PM-SYS/BE -- Project & Budget Tracking System",
    description="Enterprise-grade project management and budget tracking backend",
    version="2.0.0",
    docs_url="/api/docs",
    redoc_url="/api/redoc",
    openapi_url="/api/v1/openapi.json",
)
```

This creates the main FastAPI application.

Documentation URLs:

```
Swagger UI: /api/docs
ReDoc:      /api/redoc
OpenAPI:    /api/v1/openapi.json
```

12.2 Including the Department router

```
app.include_router(department_router)
```

This attaches every endpoint declared on `department_router` to the application.

Without this line, the controller file may exist, but FastAPI will not expose its endpoints.

Think of it as plugging the module router into the main app.

12.3 CORS

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)
```


This allows a frontend running at:

```
http://localhost:5173
```

to call the backend from a browser.

CORS is mainly a browser security concern. Postman is usually not restricted by browser CORS rules.

12.4 Health endpoint

```
@app.get("/health", tags=["Health"])
async def health_check():
    return {
        "status": "ok",
        "service": "pm-sys-backend",
        "version": "2.0.0"
    }
```

This endpoint verifies that the API process is running.

It does not necessarily prove that PostgreSQL is reachable unless it performs a database query.

12.5 Placeholder controller scanner

```
def register_controllers(app: FastAPI):
    pass
```

This function currently does nothing.

The Department router is registered manually using:

```
app.include_router(department_router)
```

The comments describe a future dynamic controller scanner similar to Spring Boot component scanning, but no scanning logic is implemented yet.

Manual router registration is explicit and often easier to debug.

13. Meaning of important Python syntax

13.1 Colon in a parameter

```
db: Session
```

The colon introduces a type annotation.

```
db          -> variable name  
Session     -> expected type
```

13.2 Equals sign in a parameter

```
db: Session = Depends(get_db)
```

The equals sign supplies a default expression.

FastAPI recognizes the `Depends` marker and resolves the dependency.

13.3 Arrow

```
-> DBDepartment
```

Expected return type annotation.

13.4 Decorator

```
@router.post(...)
```

A decorator receives or transforms the function below it. FastAPI uses it to register route metadata.

13.5 Class inheritance

```
class DepartmentDTO(BaseModel):
```

`DepartmentDTO` inherits Pydantic behavior from `BaseModel`.

```
class DBDepartment(DBDepartmentBase):
```

`DBDepartment` inherits SQLAlchemy mapping and the UUID column from `DBDepartmentBase`.

13.6 `yield`

```
yield db
```

`yield` temporarily returns a value while keeping the generator function suspended.

After the request ends, the function continues after `yield`, allowing cleanup in `finally`.

13.7 Lambda

```
default=lambda: datetime.now(timezone.utc)
```

A lambda is a small anonymous function.

It delays datetime creation until a department row is created.

13.8 Attribute access

```
department_data.dname
```

The dot accesses an object attribute.

13.9 Keyword arguments

```
DBDepartment(  
    department_name=department_data.dname  
)
```

`department_name=` is a named or keyword argument.

It makes the mapping explicit.

14. What `db.add`, `db.commit`, and `db.refresh` do

These three lines are central:

```
db.add(new_department)
db.commit()
db.refresh(new_department)
```

14.1 `db.add(new_department)`

This attaches the ORM object to the session.

It tells SQLAlchemy:

Track this object and insert it into the database during the transaction.

It does not necessarily execute the final `INSERT` immediately.

The object becomes pending.

14.2 `db.commit()`

This commits the current transaction.

Before committing, SQLAlchemy flushes pending changes, generating SQL similar to:

```
INSERT INTO departments.departments
(
    department_id,
    department_uuid,
    department_name,
    department_budget,
    department_created_at
)
VALUES
(
    ...,
    ...,
    ...,
    ...,
    ...
);
```

A successful commit makes the change permanent.

If commit fails because of a constraint or connection problem, an exception is raised.

After a failed commit, the session should usually be rolled back:

```
db.rollback()
```

14.3 `db.refresh(new_department)`

This reloads the row from the database into the existing ORM object.

It is useful for retrieving values generated by the database or ORM process, such as:

- Auto-incremented ID
- UUID
- Timestamp
- Trigger-generated data
- Server defaults

Conceptually, it performs a query similar to:

```
SELECT ...  
FROM departments.departments  
WHERE department_id = ...;
```

14.4 Difference between `flush` and `commit`

```
db.flush()
```

sends pending SQL to the database inside the current transaction but does not permanently commit it.

```
db.commit()
```

finalizes the transaction.

A service-managed transaction may use:

```
db.add(new_department)  
db.flush()  
db.refresh(new_department)
```

and commit at a higher layer. Your current repository commits directly, which is simpler but gives the service less control over multi-step transactions.

15. Detailed execution example

Assume the client sends:

```
{
  "did": 101,
  "dname": "Information Technology",
  "dbudget": 2500000.00
}
```

Phase 1: FastAPI route matching

FastAPI matches:

```
POST /v1/department/add-department
```

to:

```
create_department(...)
```

Phase 2: Pydantic validation

FastAPI creates conceptually:

```
DepartmentDTO(
  did=101,
  dname="Information Technology",
  dbudget=Decimal("2500000.00")
)
```

This object becomes `department_data`.

Phase 3: Dependency injection

FastAPI executes:

```
db = SessionLocal()
```

inside `get_db`.

Then:

```
yield db
```

injects the session into the endpoint.

Phase 4: Controller

The controller executes:

```
new_department = DepartmentService.add_department(  
    db,  
    department_data  
)
```

Phase 5: Service

The service executes:

```
return DepartmentRepo.add_department(  
    db,  
    department_data  
)
```

Phase 6: Repository object construction

The repository creates:

```
new_department = DBDepartment(  
    department_id=101,  
    department_name="Information Technology",  
    department_budget=Decimal("2500000.00")  
)
```

The model defaults produce:

- A UUID
- A UTC creation timestamp

Phase 7: Session tracking

```
db.add(new_department)
```

SQLAlchemy starts tracking the object.

Phase 8: Database transaction

```
db.commit()
```

SQLAlchemy inserts and commits the row.

Phase 9: Reloading generated values

```
db.refresh(new_department)
```

The object is refreshed from PostgreSQL.

Phase 10: Return through layers

Repository:

```
return new_department
```

Service:

```
return DepartmentRepo.add_department(...)
```

Controller receives it as:

```
new_department
```

Phase 11: HTTP response

Controller returns:

```
{"message": "Department Added Successfully"}
```

FastAPI sends:

```
HTTP/1.1 201 Created  
Content-Type: application/json
```

```
{  
  "message": "Department Added Successfully"  
}
```

Phase 12: Session cleanup

After the endpoint finishes, `get_db` resumes and executes:

```
db.close()
```


16. Spring Boot comparison

FastAPI/Python	Spring Boot/Java equivalent
<code>FastAPI()</code>	<code>@SpringBootApplication</code> application
<code>APIRouter</code>	<code>@RestController</code> + class-level mapping
<code>@router.post(...)</code>	<code>@PostMapping(...)</code>
Pydantic <code>BaseModel</code>	DTO class/record
<code>department_data: DepartmentDTO</code>	<code>@RequestBody</code> <code>DepartmentDTO</code> <code>departmentData</code>
SQLAlchemy model	JPA <code>@Entity</code>
<code>Session</code>	<code>EntityManager</code> / repository transaction context
Repository class	<code>@Repository</code>
Service class	<code>@Service</code>
<code>Depends(get_db)</code>	Dependency injection/lifecycle-managed resource
<code>HTTPException</code>	<code>ResponseStatusException</code> or exception handler
<code>app.include_router(router)</code>	Component discovery and controller registration
<code>db.add</code>	<code>entityManager.persist</code> or repository <code>save</code>
<code>db.commit</code>	Transaction commit
<code>db.refresh</code>	<code>entityManager.refresh</code>
<code>-> DBDepartment</code>	Java return type before method name

Comparison:

Python:

```
def add_department(  
    db: Session,  
    department_data: DepartmentDTO  
) -> DBDepartment:
```

Java:

```
public DBDepartment addDepartment(  
    EntityManager db,  
    DepartmentDTO departmentData  
) {  
    ...  
}
```

In Java, the return type appears before the method name.

In Python, the return type annotation appears after `->`.

17. Problems and improvement opportunities in the uploaded code

The code passes Python syntax compilation, but several architectural and runtime concerns should be addressed.

17.1 Duplicate imports in the controller

The controller imports FastAPI names twice:

```
from fastapi import APIRouter, Depends, status
```

and:

```
from fastapi import APIRouter, Depends, HTTPException, status
```

Keep one:

```
from fastapi import APIRouter, Depends, status
```

Add `HTTPException` only when it is used.

17.2 Duplicate and unused imports in the model

The model imports `Column` and `DateTime` twice and imports `datetime` in two forms.

Current pattern:

```
import datetime
from datetime import datetime, timezone
```

The first `import datetime` is unused.

`Column` and `DateTime` are also imported twice.

Cleaner:

```
from datetime import datetime, timezone
import uuid as py_uuid

from sqlalchemy import UUID, Column, DateTime, Integer, Numeric, String
```

17.3 Unused imports

Examples:

- `or_` in `DepartmentRepo.py`
- `HTTPException` and `status` in `DepartmentService.py`
- `datetime` in `DepartmentSchema.py`
- `importlib`, `pkgutil`, and `APIRouter` in `main.py`

Unused imports create noise and may mislead future developers.

17.4 Import path inconsistency

Files import:

```
from core.database import Base
```

and:

```
from core.database import get_db
```

But the uploaded file is named `database.py` at the upload root.

The final project must place it consistently, for example:

```
core/database.py
```

Then the import is correct:

```
from core.database import Base, get_db
```

Alternatively, if it is:

```
core/database/database.py
```

the import would be:

```
from core.database.database import Base, get_db
```

Choose one structure and use it everywhere.

17.5 Module path assumption

The archive contains a top-level `departments` directory, while imports use:

```
modules.departments...
```

Therefore the extracted folder must be placed at:

```
project_root/modules/departments
```

Otherwise Python raises:

```
ModuleNotFoundError: No module named 'modules'
```

17.6 Auto-increment ID versus DTO-provided ID

The model says:

```
autoincrement=True
```

but the request requires `did` and the repository sets `department_id`.

Recommended: remove ID from the create DTO and let PostgreSQL generate it.

17.7 No response model

The endpoint returns only a message.

A better response provides created data and documents its shape:

```
@router.post(
    "",
    response_model=DepartmentResponse,
    status_code=status.HTTP_201_CREATED
)
```

This makes Swagger documentation more precise and prevents accidental exposure of unwanted model fields.

17.8 No rollback on commit failure

Current:

```
db.commit()
```

If this fails, the session enters a failed transaction state.

Safer:

```
try:
    db.add(new_department)
    db.commit()
    db.refresh(new_department)
    return new_department
except Exception:
    db.rollback()
    raise
```

For production code, catch known SQLAlchemy exceptions where possible rather than converting every exception into the same message.

17.9 No duplicate-department check

The service imports `HTTPException` but does not perform validation.

A future service can check:

```
existing = DepartmentRepo.get_by_name(db, department_data.department_name)
if existing:
    raise HTTPException(
        status_code=status.HTTP_409_CONFLICT,
        detail="Department already exists"
    )
```

The database should also enforce uniqueness if uniqueness is a real invariant.

Service validation alone is not enough under concurrent requests.

17.10 Department name nullable

The database column does not specify:

```
nullable=False
```

For a required name, add it.

17.11 Budget validation

Pydantic should reject negative budgets:

```
department_budget: Decimal = Field(ge=0)
```

The database may also use a check constraint.

17.12 Synchronous versus asynchronous description

The `main.py` description says:

```
SQLAlchemy 2.0 (async ORM)
```

but the current code is synchronous.

Either:

- Change the documentation to "synchronous ORM," or
- Convert the engine/session/repository/controller to async.

Do not mix sync and async terminology because it confuses maintainers.

17.13 Database schema creation

The model targets:

```
departments.departments
```

The PostgreSQL schema `departments` must exist.

Alembic migration example:

```
op.execute("CREATE SCHEMA IF NOT EXISTS departments")
```

Then create the table.

17.14 `create_all` is commented out

The uploaded `main.py` comments out:

```
Base.metadata.create_all(bind=engine)
```

Therefore importing the app alone will not create the table.

That is acceptable if Alembic migrations are the intended method.

For a serious project, prefer Alembic over automatic `create_all`.

17.15 Controller variable not used

```
new_department = ...
```

but only a message is returned.

Either use it in the response or do not assign it:

```
DepartmentService.add_department(db, department_data)
```

Returning created data is generally more useful.

17.16 Route naming

Current:

```
POST /v1/department/add-department
```

A more REST-style route is:

```
POST /api/v1/departments
```

The HTTP method already expresses "add/create," so the path does not need a verb.

This is a design recommendation, not a requirement.

18. A cleaner learning-oriented implementation

The following example keeps the same layered architecture while improving naming, validation, transactions, and response handling.

18.1 `core/database.py`

```
import os
from collections.abc import Generator

from dotenv import load_dotenv
from sqlalchemy import URL, create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

load_dotenv()

DATABASE_URL = URL.create(
    drivename="postgresql+psycopg2",
    username=os.getenv("DB_USER", "postgres"),
    password=os.getenv("DB_PASSWORD"),
    host=os.getenv("DB_HOST", "localhost"),
    port=int(os.getenv("DB_PORT", "5432")),
    database=os.getenv("DB_NAME", "project_management"),
)

engine = create_engine(
    DATABASE_URL,
    pool_pre_ping=True,
)

SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

18.2 `models/DepartmentTables.py`


```
from datetime import datetime, timezone
from decimal import Decimal
import uuid as py_uuid

from sqlalchemy import DateTime, Integer, Numeric, String, UUID
from sqlalchemy.orm import Mapped, mapped_column

from core.database import Base

class DBDepartment(Base):
    __tablename__ = "departments"
    __table_args__ = {"schema": "departments"}

    department_id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        autoincrement=True,
    )

    department_uuid: Mapped[py_uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        default=py_uuid.uuid4,
        unique=True,
        nullable=False,
        index=True,
    )

    department_name: Mapped[str] = mapped_column(
        String(150),
        nullable=False,
        unique=True,
        index=True,
    )

    department_budget: Mapped[Decimal] = mapped_column(
        Numeric(15, 2),
        nullable=False,
        default=Decimal("0.00"),
    )

    department_created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
```

This uses SQLAlchemy 2.0's typed `Mapped` style.

18.3 `schemas/DepartmentSchema.py`

```
from datetime import datetime
from decimal import Decimal
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field

class DepartmentCreate(BaseModel):
    department_name: str = Field(
        min_length=2,
        max_length=150,
    )
    department_budget: Decimal = Field(
        ge=0,
        max_digits=15,
        decimal_places=2,
    )

class DepartmentResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    department_id: int
    department_uuid: UUID
    department_name: str
    department_budget: Decimal
    department_created_at: datetime

class DepartmentCreatedResponse(BaseModel):
    message: str
    data: DepartmentResponse
```

Why separate create and response schemas?

Input should not necessarily have every database field.

The client sends:

```
name and budget
```

The server generates:

```
ID, UUID, and creation time
```

Therefore:

```
DepartmentCreate -> incoming request
DepartmentResponse -> outgoing data
```

18.4 [repositories/DepartmentRepo.py](#)

```
from sqlalchemy import select
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.orm import Session

from modules.departments.models.DepartmentTables import DBDepartment
from modules.departments.schemas.DepartmentSchema import DepartmentCreate

class DepartmentRepo:
    @staticmethod
    def get_by_name(
        db: Session,
        department_name: str,
    ) -> DBDepartment | None:
        statement = select(DBDepartment).where(
            DBDepartment.department_name == department_name
        )
        return db.scalar(statement)

    @staticmethod
    def add_department(
        db: Session,
        department_data: DepartmentCreate,
    ) -> DBDepartment:
        new_department = DBDepartment(
            department_name=department_data.department_name.strip(),
            department_budget=department_data.department_budget,
        )

        try:
            db.add(new_department)
            db.commit()
            db.refresh(new_department)
            return new_department
        except SQLAlchemyError:
            db.rollback()
            raise
```

Meaning of `DBDepartment | None`

This means the function may return:

DBDepartment

or:

None

It is Python's modern union type syntax.

18.5 `services/DepartmentService.py`

```
from fastapi import HTTPException, status
from sqlalchemy.orm import Session

from modules.departments.models.DepartmentTables import DBDepartment
from modules.departments.repositories.DepartmentRepo import DepartmentRepo
from modules.departments.schemas.DepartmentSchema import DepartmentCreate

class DepartmentService:
    @staticmethod
    def add_department(
        db: Session,
        department_data: DepartmentCreate,
    ) -> DBDepartment:
        normalized_name = department_data.department_name.strip()

        existing_department = DepartmentRepo.get_by_name(
            db,
            normalized_name,
        )

        if existing_department is not None:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="A department with this name already exists",
            )

        return DepartmentRepo.add_department(
            db,
            department_data,
        )
```

18.6 controllers/DepartmentController.py

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session

from core.database import get_db
from modules.departments.schemas.DepartmentSchema import (
    DepartmentCreate,
    DepartmentCreatedResponse,
)
from modules.departments.services.DepartmentService import DepartmentService

router = APIRouter(
    prefix="/api/v1/departments",
    tags=["Departments"],
)

@router.post(
    "",
    response_model=DepartmentCreatedResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_department(
    department_data: DepartmentCreate,
    db: Session = Depends(get_db),
) -> DepartmentCreatedResponse:
    new_department = DepartmentService.add_department(
        db,
        department_data,
    )

    return DepartmentCreatedResponse(
        message="Department added successfully",
        data=new_department,
    )
```

FastAPI can create the nested response because `DepartmentResponse` uses:

```
ConfigDict(from_attributes=True)
```

This permits Pydantic to read attributes from a SQLAlchemy object.

18.7 `main.py`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from modules.departments.controllers.DepartmentController import (
    router as department_router,
)

app = FastAPI(
    title="PM-SYS/BE -- Project & Budget Tracking System",
    version="2.0.0",
    docs_url="/api/docs",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(department_router)

@app.get("/health", tags=["Health"])
def health_check() -> dict[str, str]:
    return {
        "status": "ok",
        "service": "pm-sys-backend",
        "version": "2.0.0",
    }
```

19. How to create another module in the future

Suppose the next module is `projects`.

Use the same process.

Step 1: Identify the database fields

Example:

```
project_id
project_uuid
project_name
project_budget
project_status
project_created_at
```

Decide:

- Required fields
- Optional fields
- Primary key
- Unique columns
- Data types
- Relationships
- Constraints
- Defaults

Step 2: Create the model

File:

```
modules/projects/models/ProjectTables.py
```

The model defines the database representation.

Step 3: Create schemas

File:

```
modules/projects/schemas/ProjectSchema.py
```

Prefer multiple schemas:

```
ProjectCreate
ProjectUpdate
ProjectResponse
ProjectListResponse
```

Do not force one DTO to serve every operation.

Step 4: Create repository

File:

```
modules/projects/repositories/ProjectRepo.py
```

Repository methods may include:

```
add_project
get_project_by_id
get_project_by_uuid
get_project_by_name
list_projects
update_project
delete_project
```

Step 5: Create service

File:

```
modules/projects/services/ProjectService.py
```

Add business rules:

- Prevent duplicates
- Validate budget
- Validate department existence
- Check permissions
- Handle allowed status transitions

Step 6: Create controller

File:

```
modules/projects/controllers/ProjectController.py
```

Define HTTP endpoints:


```
POST /api/v1/projects
GET /api/v1/projects
GET /api/v1/projects/{project_id}
PATCH /api/v1/projects/{project_id}
DELETE /api/v1/projects/{project_id}
```

Step 7: Register router

In `main.py`:

```
app.include_router(project_router)
```

Step 8: Create migration

Use Alembic to generate and review a migration.

Conceptual commands:

```
alembic revision --autogenerate -m "create projects table"
alembic upgrade head
```

Always inspect generated migrations before applying them.

Step 9: Test every layer

Write:

- Schema validation tests
- Repository tests
- Service business-rule tests
- Controller/API tests

20. Testing checklist

20.1 Successful creation

Request:

```
{
  "department_name": "Finance",
  "department_budget": 500000.00
}
```

Expected:

```
HTTP 201
```

Database contains one new department.

20.2 Missing name

Request:

```
{
  "department_budget": 500000.00
}
```

Expected:

```
HTTP 422
```

20.3 Negative budget

Request:

```
{
  "department_name": "Finance",
  "department_budget": -1
}
```

Expected:

```
HTTP 422
```

when Pydantic `Field(ge=0)` is used.

20.4 Duplicate name

Create `Finance` twice.

Expected for the second request:

```
HTTP 409 Conflict
```

20.5 Database unavailable

Stop PostgreSQL and call the endpoint.

Expected:

- Controlled server error
- Transaction safely handled
- Useful logging
- No sensitive database credentials exposed

20.6 Generated fields

Verify that the response/database row contains:

- Generated ID
- UUID
- UTC creation timestamp

20.7 Session cleanup

Ensure sessions are closed after successful and failed requests.

21. Common errors and how to understand them

`ModuleNotFoundError: No module named 'modules'`

Cause:

- Running Python from the wrong working directory
- Missing package structure
- Department folder not under `modules`
- Incorrect import path

Verify:

```
project_root/  
|-- main.py  
+-- modules/  
    |-- departments/
```

Run Uvicorn from `project_root`.

`ModuleNotFoundError: No module named 'core'`

Cause:

- `database.py` is not inside `core`
- Import path does not match folder structure

`relation "departments.departments" does not exist`

Cause:

- Migration not applied
- Table not created
- Wrong schema
- Wrong database

`schema "departments" does not exist`

Create the PostgreSQL schema through migration:

```
CREATE SCHEMA IF NOT EXISTS departments;
```

duplicate key value violates unique constraint

Cause:

- Duplicate primary key
- Duplicate UUID
- Duplicate unique department name

In the uploaded code, manually supplying `did` can cause duplicate primary-key errors.

current transaction is aborted

Cause:

A database operation failed and the session was reused without:

```
db.rollback()
```

422 Unprocessable Entity

Cause:

The request body does not match the Pydantic schema.

Check:

- Field names
- Required fields
- Data types
- JSON syntax
- Validation constraints

500 Internal Server Error

This is a general server failure. Check the backend traceback.

Do not expose full tracebacks to production clients.

22. Final mental model

Remember this story:

```
The client sends JSON.
FastAPI matches the route.
Pydantic validates the JSON and creates a DTO object.
Depends(get_db) creates a SQLAlchemy session.
The controller receives the DTO and session.
The controller calls the service.
The service applies business rules.
The service calls the repository.
The repository creates a SQLAlchemy model object.
db.add tracks it.
db.commit saves it permanently.
db.refresh reloads generated values.
The object returns through repository -> service -> controller.
FastAPI converts the response into JSON.
get_db closes the session.
```

The essential code chain is:

```
# Controller
DepartmentService.add_department(db, department_data)
```

```
# Service
DepartmentRepo.add_department(db, department_data)
```

```
# Repository
new_department = DBDepartment(...)
db.add(new_department)
db.commit()
db.refresh(new_department)
return new_department
```

And the key meaning of:

```
-> DBDepartment
```

is:

The function is expected to return a `DBDepartment` object.

It is documentation and type information, while:

```
return new_department
```

is the statement that actually returns the object.

Recommended development sequence summary

For every future feature module:

1. Understand the requirement
2. Design database fields and constraints
3. Create/update database foundation
4. Create SQLAlchemy model
5. Create request/response schemas
6. Create repository methods
7. Create service rules
8. Create controller endpoints
9. Register router in main.py
10. Create and apply migration
11. Test successful and failure cases
12. Add logging, authorization, and documentation

This sequence helps you think like the framework instead of merely copying code.